Lab : Numerical and process-based modelling

Peta Koloamatangi (686891371)

2026-08-10

Table of contents

Setup 1
Part A: Tracer decay and step size (numerical) 1
Task 1 2
Task 2 4
Task 3 6
Task 4: 8
Part B: Reservoir box model (process-based) 8
Task 1 8
Task 2 10
Task 3 12
Task 4 13
Reflection 15

Setup

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Part A: Tracer decay and step size (numerical)

This part illustrates modelling the decay of a tracer in a system using numerical methods and comparing it to the exact solution.

Task 1

decay rate = 0.1

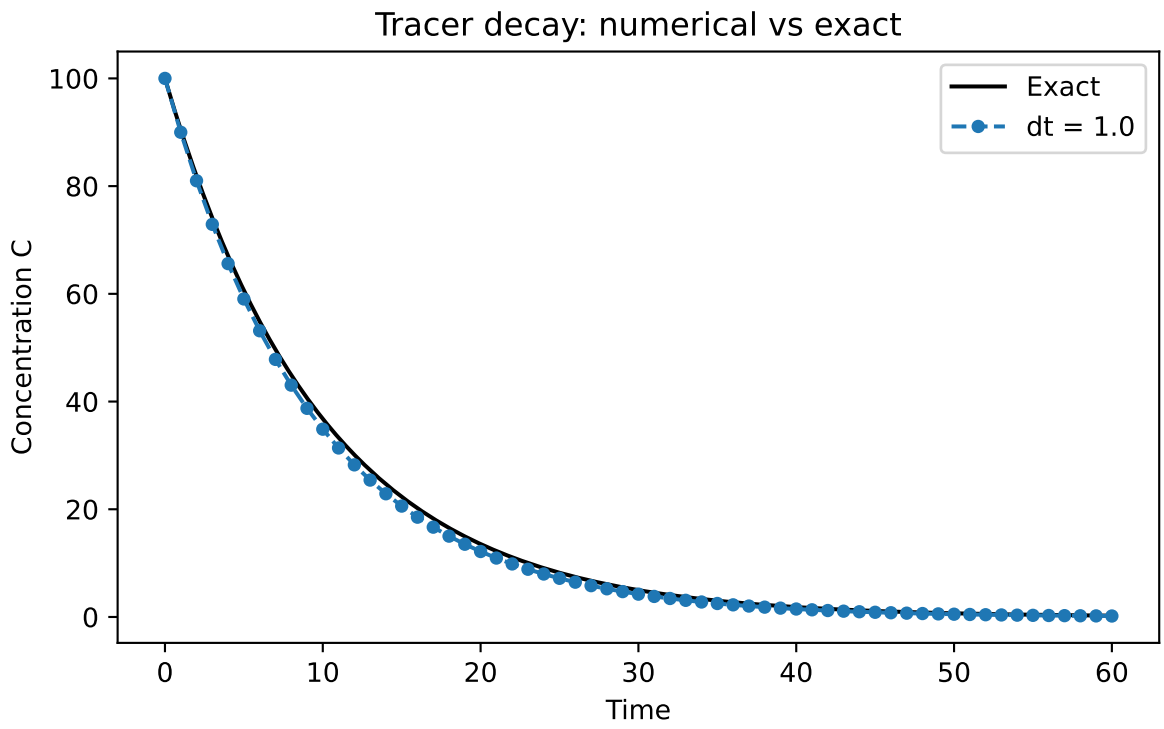
```
def decay_numerical(C0, k, dt, t_end):
    """Return arrays of time and concentration for forward-Euler tracer decay."""
    n = int(t_end / dt)
    C = C0
    ts = [0.0]
    Cs = [C0]
    for i in range(n):
        C = C + (-k * C) * dt          # forward difference
        ts.append((i + 1) * dt)
        Cs.append(C)
    return np.array(ts), np.array(Cs)

# The exact answer, for comparison
C0, k, t_end = 100.0, 0.1, 60
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_fine, exact, "k-", label="Exact")

for dt in [1.0]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("Tracer decay: numerical vs exact")
ax.legend()
plt.show()
```



decay rate = 0.01

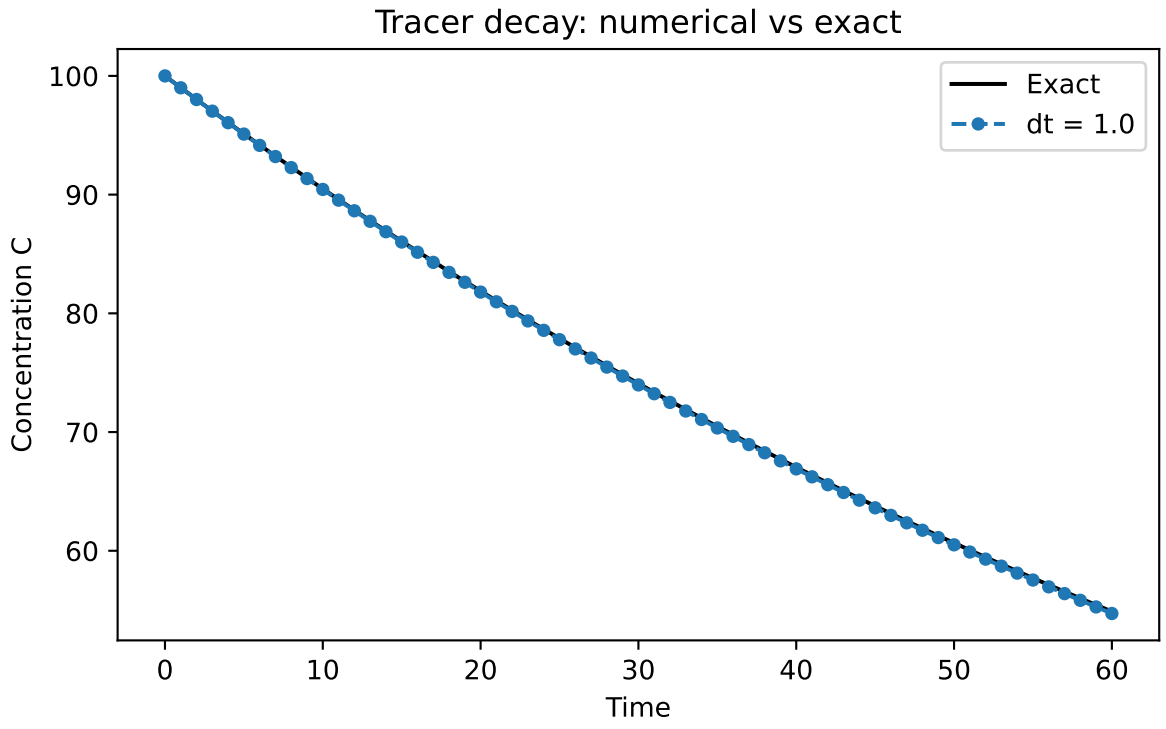
```
def decay_numerical(C0, k, dt, t_end):
    """Return arrays of time and concentration for forward-Euler tracer decay."""
    n = int(t_end / dt)
    C = C0
    ts = [0.0]
    Cs = [C0]
    for i in range(n):
        C = C + (-k * C) * dt          # forward difference
        ts.append((i + 1) * dt)
        Cs.append(C)
    return np.array(ts), np.array(Cs)

# The exact answer, for comparison
C0, k, t_end = 100.0, 0.01, 60
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_fine, exact, "k-", label="Exact")
```

```
for dt in [1.0]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("Tracer decay: numerical vs exact")
ax.legend()
plt.show()
```



Answer to task A1: The decay rate controls the speed of tracer decay; a lower decay rate (e.g., k = 0.01) would gradually decrease the concentration over time compared to a higher decay rate like k = 0.1.

Task 2

dt = 2.0, 5.0

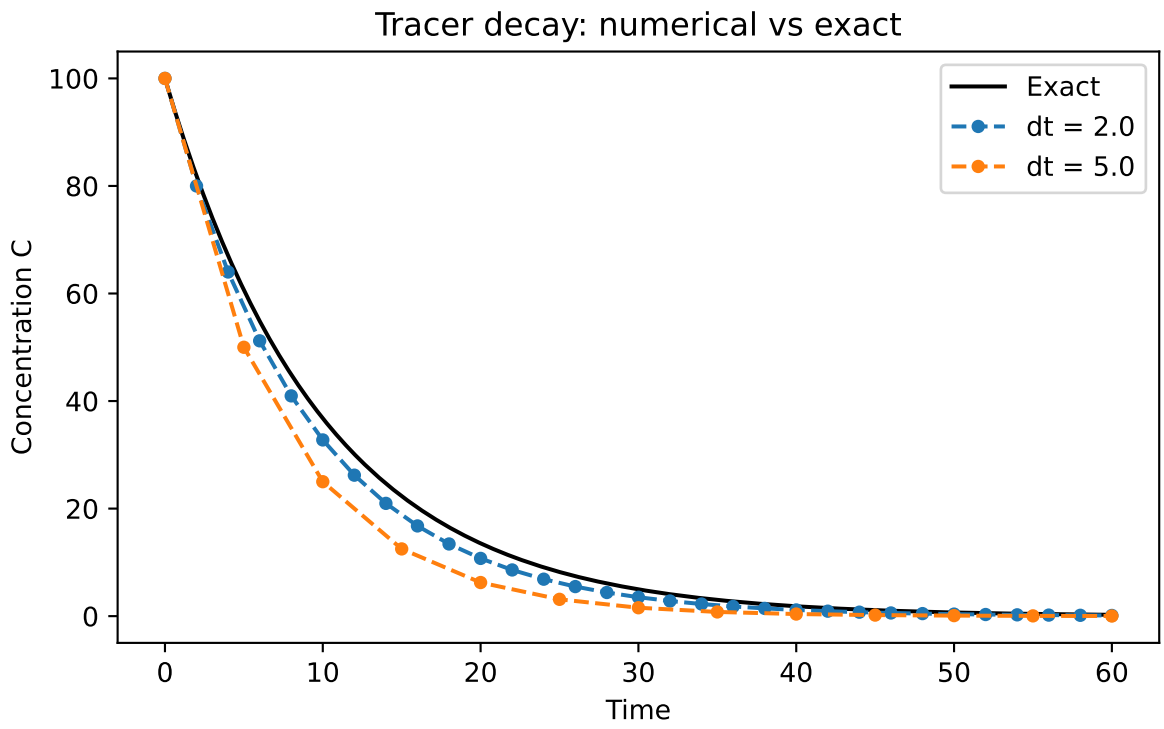
```
def decay_numerical(C0, k, dt, t_end):
    """Return arrays of time and concentration for forward-Euler tracer decay."""
    n = int(t_end / dt)
    C = C0
    ts = [0.0]
    Cs = [C0]
    for i in range(n):
        C = C + (-k * C) * dt          # forward difference
        ts.append((i + 1) * dt)
        Cs.append(C)
    return np.array(ts), np.array(Cs)

# The exact answer, for comparison
C0, k, t_end = 100.0, 0.1, 60
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_fine, exact, "k-", label="Exact")

for dt in [2.0, 5.0]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("Tracer decay: numerical vs exact")
ax.legend()
plt.show()
```



Answer to task A2: The numerical curve for dt = 5.0 starts to deviate from the exact solution at around t = 10 days while that of dt = 2.0 remains closer to the exact solution.

Task 3

```
def decay_numerical(C0, k, dt, t_end):
    """Return arrays of time and concentration for forward-Euler tracer decay."""
    n = int(t_end / dt)
    C = C0
    ts = [0.0]
    Cs = [C0]
    for i in range(n):
        C = C + (-k * C) * dt          # forward difference
        ts.append((i + 1) * dt)
        Cs.append(C)
    return np.array(ts), np.array(Cs)

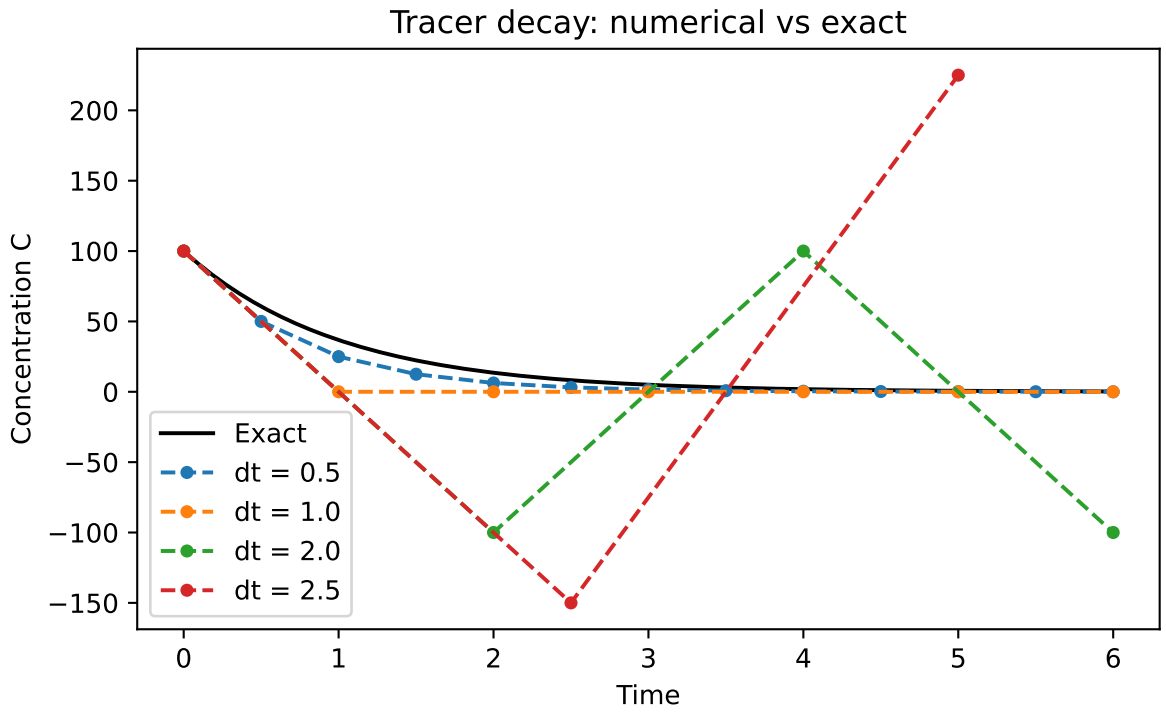
# The exact answer, for comparison
C0, k, t_end = 100.0, 1.0, 6.0
```

```
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_fine, exact, "k-", label="Exact")

for dt in [0.5, 1.0, 2.0, 2.5]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("Tracer decay: numerical vs exact")
ax.legend()
plt.show()
```



Answer to task A3: 1.0 is the largest dt that gives a non-negative curve.

Task 4:

Answer to task A4: Increased dt will cause the numerical solution to diverge from the exact solution. The update factor 1−k dt1 - k,dt1−kdt from the update rule becomes a negative factor that causes the numerical solution to go past zero, leading to negative concentrations and instability in the model. This is illustrated in the plot above, where dt = 2.5 results in a negative concentration after a few time steps.

Part B: Reservoir box model (process-based)

Task 1

```
import numpy as np
import matplotlib.pyplot as plt

# ---- Parameters ----
V = 1.0e6        # lake volume (m^3)
Q = 5.0e3        # river inflow (m^3 per day)
C_in = 10.0      # incoming concentration (mg/L)
k = 0.05         # in-lake decay rate (per day)

# ---- Time settings ----
dt = 1.0         # time step (days)
days = 200

# ---- Initial condition ----
C = 0.0          # clean lake

# ---- Storage ----
history = [C]

# ---- Time loop ----
for _ in range(days):
    dCdt = (Q * C_in - Q * C - k * V * C) / V   # the rate of change
    C = C + dCdt * dt                           # forward difference
    history.append(C)

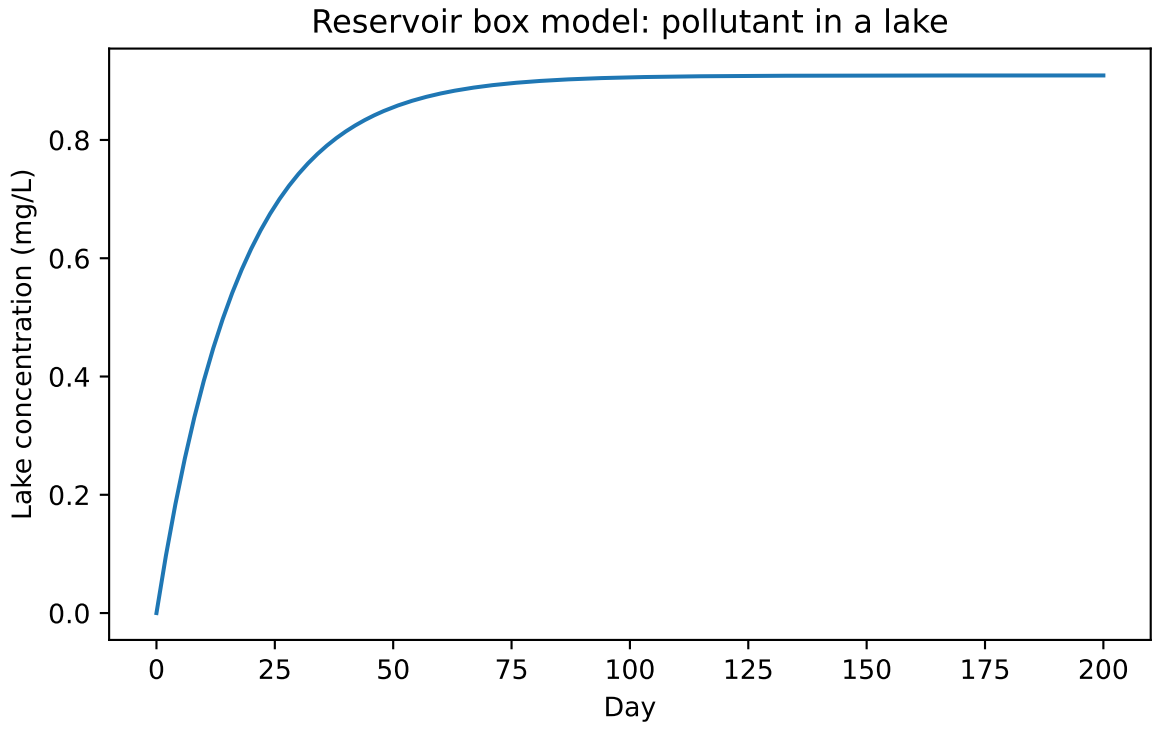
# ---- Analytical steady state ----
C_star = (Q * C_in) / (Q + k * V)

# ---- A quick look at the numbers ----
```

```
print(f"Started at {history[0]:.2f} mg/L")
print(f"After 10 days: {history[10]:.2f} mg/L")
print(f"After 50 days: {history[50]:.2f} mg/L")
print(f"After {days} days: {history[-1]:.2f} mg/L")

# --- Plotting ----
fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(history)
ax.set_xlabel("Day")
ax.set_ylabel("Lake concentration (mg/L)")
ax.set_title("Reservoir box model: pollutant in a lake")
plt.show()
```

```
Started at 0.00 mg/L
After 10 days: 0.39 mg/L
After 50 days: 0.86 mg/L
After 200 days: 0.91 mg/L
```



Confirming the analytical steady state:

```
C_star = (Q * C_in) / (Q + k * V)
print(f"Analytical steady state: {C_star:.2f} mg/L")
print(f"Numerical final value:   {history[-1]:.2f} mg/L")
print(f"Difference:              {abs(C_star - history[-1]):.4f} mg/L")
```

```
Analytical steady state: 0.91 mg/L
Numerical final value:   0.91 mg/L
Difference:              0.0000 mg/L
```

Answer to task B1: Analytical steady state: 0.91 mg/L Numerical final value: 0.91 mg/L

Task 2

```
import numpy as np
import matplotlib.pyplot as plt

# ---- Parameters ----
V = 1.0e6        # lake volume (m^3)
Q = 5.0e3        # river inflow (m^3 per day)
C_in = 10.0      # incoming concentration (mg/L)
k = 0.05         # in-lake decay rate (per day)

# ---- Time settings ----
dt = 1.0         # time step (days)
days = 200

# ---- Initial condition ----
C = 20.0          # clean lake

# ---- Storage ----
history = [C]

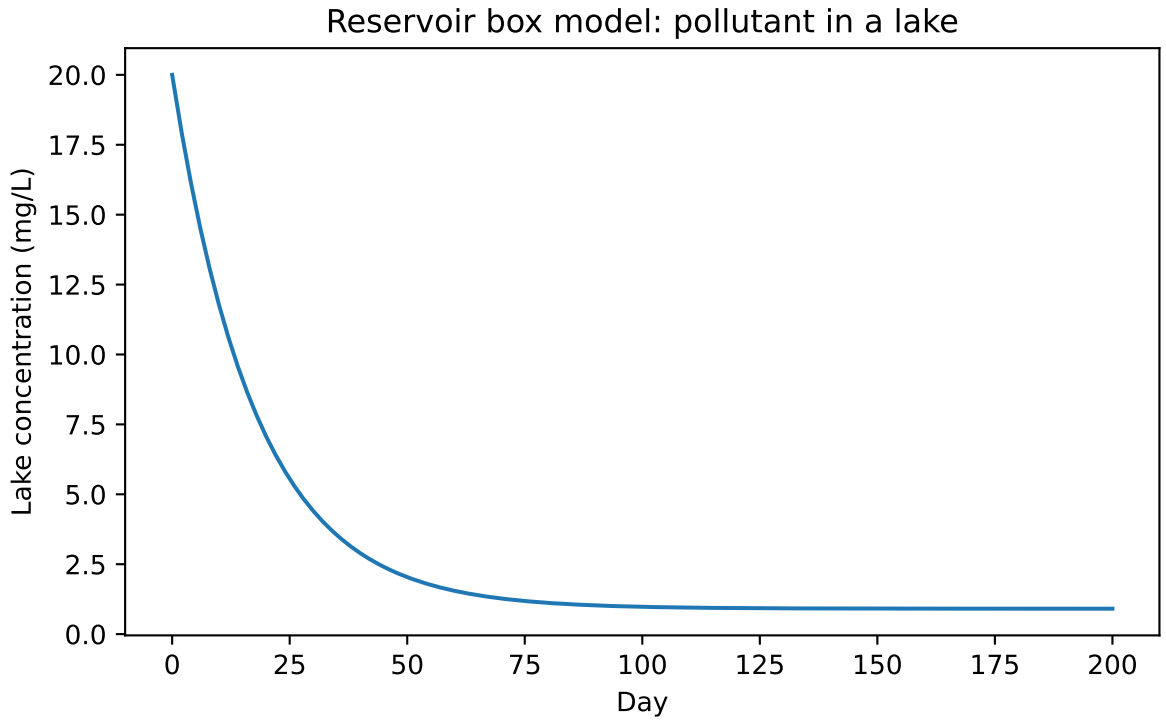
# ---- Time loop ----
for _ in range(days):
    dCdt = (Q * C_in - Q * C - k * V * C) / V   # the rate of change
    C = C + dCdt * dt                           # forward difference
    history.append(C)

# ---- A quick look at the numbers ----
```

```
print(f"Started at {history[0]:.2f} mg/L")
print(f"After 10 days: {history[10]:.2f} mg/L")
print(f"After 50 days: {history[50]:.2f} mg/L")
print(f"After {days} days: {history[-1]:.2f} mg/L")

# --- Plotting ----
fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(history)
ax.set_xlabel("Day")
ax.set_ylabel("Lake concentration (mg/L)")
ax.set_title("Reservoir box model: pollutant in a lake")
plt.show()
```

```
Started at 20.00 mg/L
After 10 days: 11.75 mg/L
After 50 days: 2.04 mg/L
After 200 days: 0.91 mg/L
```



Answer to task B2: The lake concentration starts at 20 mg/L and decreases over time, approaching the steady-state concentration of approximately 0.91 mg/L after 200 days.

Task 3

```
import numpy as np
import matplotlib.pyplot as plt

# ---- Parameters ----
V = 1.0e6        # lake volume (m^3)
Q = 10.0e3        # river inflow (m^3 per day)
C_in = 10.0      # incoming concentration (mg/L)
k = 0.05         # in-lake decay rate (per day)

# ---- Time settings ----
dt = 1.0         # time step (days)
days = 200

# ---- Initial condition ----
C = 0.0          # clean lake

# ---- Storage ----
history = [C]

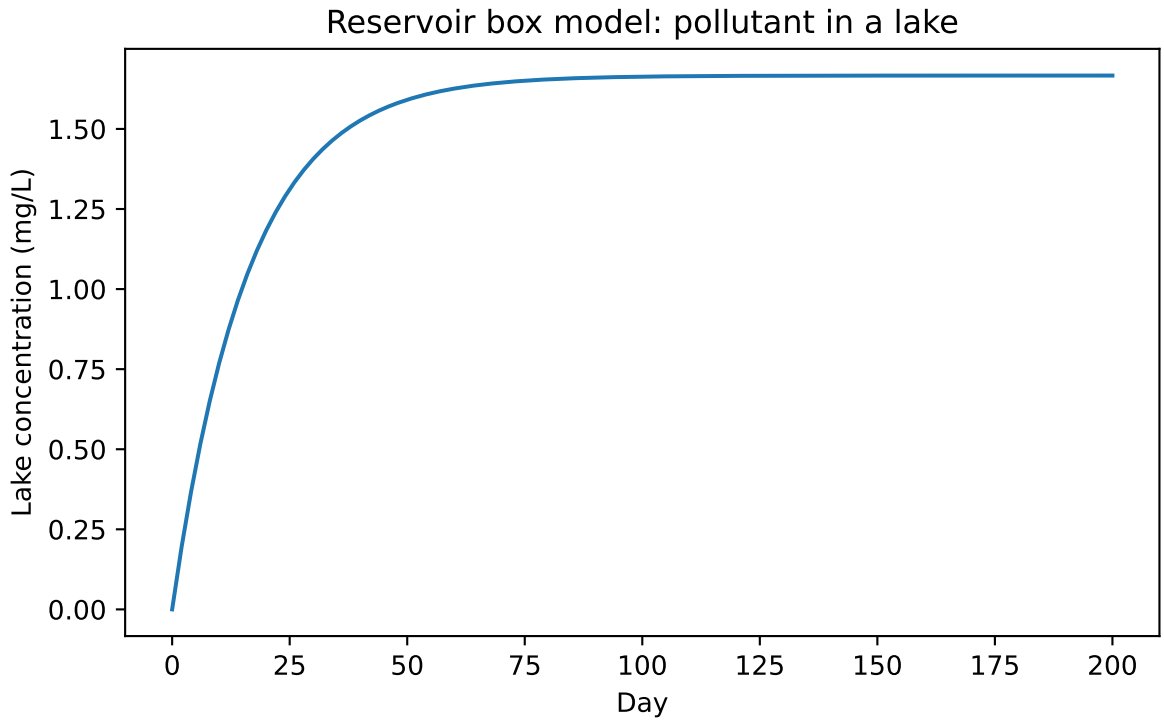
# ---- Time loop ----
for _ in range(days):
    dCdt = (Q * C_in - Q * C - k * V * C) / V   # the rate of change
    C = C + dCdt * dt                           # forward difference
    history.append(C)

# ---- A quick look at the numbers ----
print(f"Started at {history[0]:.2f} mg/L")
print(f"After 10 days: {history[10]:.2f} mg/L")
print(f"After 50 days: {history[50]:.2f} mg/L")
print(f"After {days} days: {history[-1]:.2f} mg/L")

# --- Plotting ----
fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(history)
ax.set_xlabel("Day")
ax.set_ylabel("Lake concentration (mg/L)")
ax.set_title("Reservoir box model: pollutant in a lake")
plt.show()
```

```
Started at 0.00 mg/L
```

```
After 10 days: 0.77 mg/L
After 50 days: 1.59 mg/L
After 200 days: 1.67 mg/L
```



Answer to task B3: The lake concentration starts at 0 mg/L and increases over time, approaching the steady-state concentration of approximately 1.67 mg/L after 200 days.

Task 4

```python
import numpy as np
import matplotlib.pyplot as plt

# ---- Parameters ----
V = 1.0e6        # lake volume (m^3)
Q = 5.0e3        # river inflow (m^3 per day)
C_in = 10.0      # incoming concentration (mg/L)
k = 0.05         # in-lake decay rate (per day)

# ---- Time settings ----
```

```
dt = 1.0          # time step (days)
days = 200

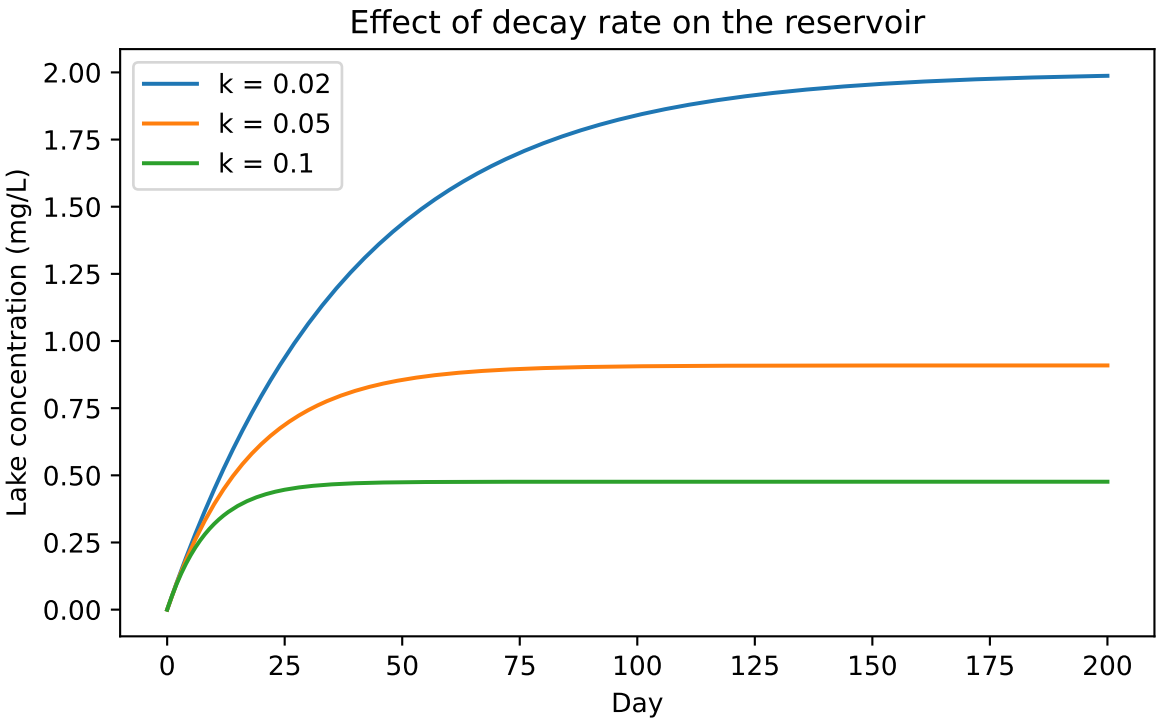
# ---- Initial condition ----
C = 0.0           # clean lake

# ---- Storage ----
history = [C]

# ---- Time loop ----
for _ in range(days):
    dCdt = (Q * C_in - Q * C - k * V * C) / V   # the rate of change
    C = C + dCdt * dt                            # forward difference
    history.append(C)
fig, ax = plt.subplots(figsize=(7, 4))

for k_test in [0.02, 0.05, 0.10]:
    C = 0.0
    history = [C]
    for _ in range(days):
        dCdt = (Q * C_in - Q * C - k_test * V * C) / V
        C = C + dCdt * dt
        history.append(C)
    ax.plot(history, label=f"k = {k_test}")

ax.set_xlabel("Day")
ax.set_ylabel("Lake concentration (mg/L)")
ax.set_title("Effect of decay rate on the reservoir")
ax.legend()
plt.show()
```



Answer to task B4: A larger decay rate (k = 0.10) removes the tracer from the reservoir more rapidly hence approaches the steady state faster.

Reflection

I spent quite a bit of time on this lab, trying to understand the concepts of reservoir modeling and the impact of different parameters on the system's behavior. For Part A, I learned how the decay rate and time step size can significantly affect the accuracy of numerical solutions compared to exact solutions. I also realized that I could have plotted just one figure for Task A1 but I didn't. In Part B, I gained insights into how inflow rates and decay rates influence the concentration of pollutants in a reservoir over time. I learned while doing Task B4 that I have to be more careful with the parameters like inflow (Q). I initially continued my coding from B3 and my figure resulted with a concentration which was more than 3mg/L so I had to insert again the whole original code from the 2.2 Example 1 to ensure that inflow rate stayed at 5.0e3. Overall, this lab helped me appreciate the importance of parameter selection in environmental modeling and the need for careful analysis when interpreting model results and also be more careful with my code.